HI-LO GAME

HI-LO GAME

- **Gioco:** Indovina un numero segreto tra 1 e 100.
- **Tentativi:** Hai un massimo di 10 tentativi.
- **Feedback:** Ad ogni tentativo, ricevi come risposta se il numero è troppo **alto** (`hi`) o troppo **basso** (`lo`).

HI-LO GAME SERVICE REQUIREMENTS

Questi requisiti funzionali sono la base per il design della nostra API:

1. **Avviare** una nuova partita.
2. **Accettare** un tentativo (fino a 10 tentativi), e restituire `hi`, `lo`, o `correct`.
3. **Resettare** una partita, generando un nuovo numero segreto.
4. **Ottenere** la lista di ogni tentativo in una partita, con i risultati correlati.
5. **Ottenere** la lista di tutte le partite, con il risultato finale (`win`/`lose`) e il numero di tentativi.

Che tipo di risorse progetteresti?

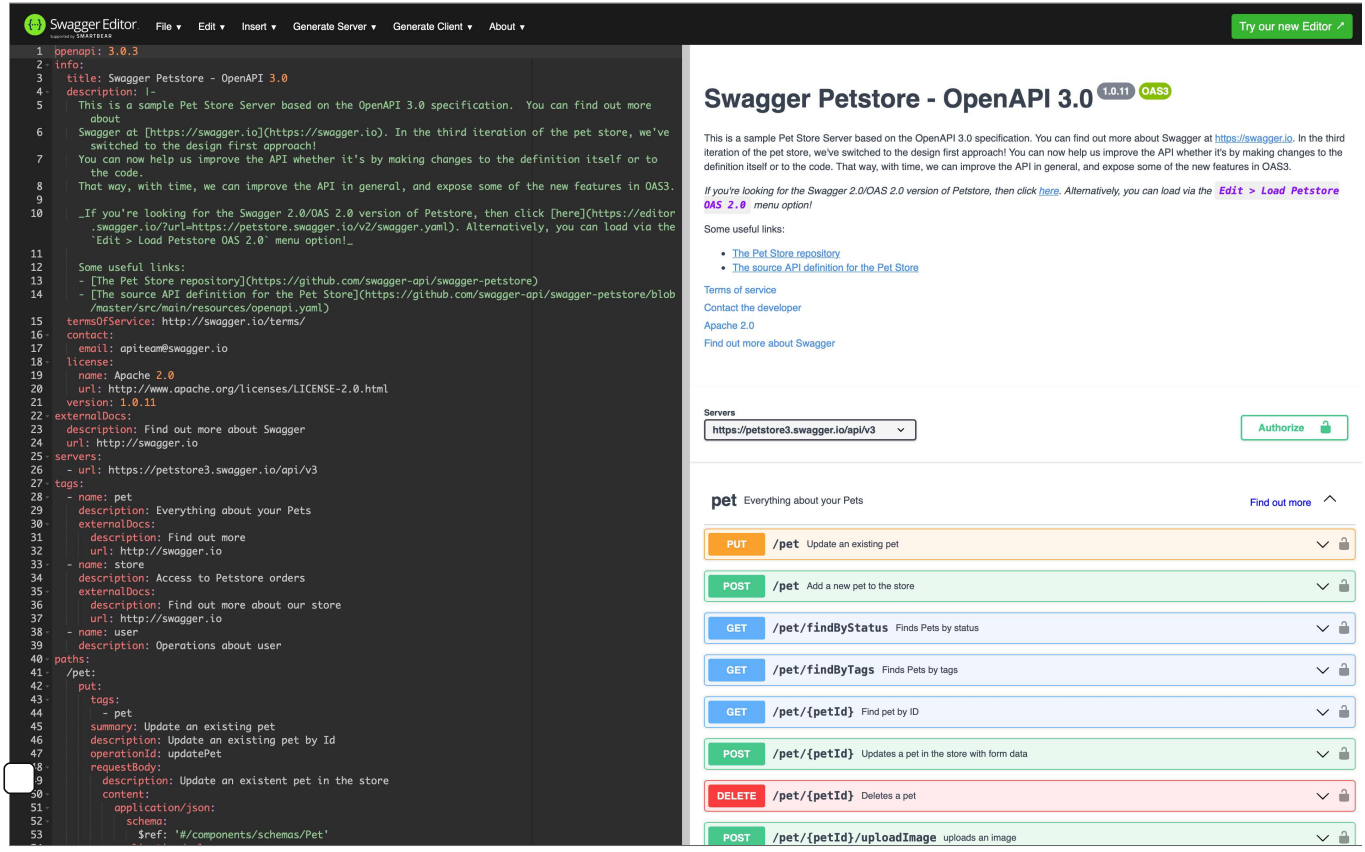
PARTE I: DESIGN DEL SERVIZIO

Il design dell'API segue i principi **REST (Representational State Transfer)**, mappando i requisiti ai concetti di **Risorse (sostantivi)** e **Operazioni (verbi HTTP)**.

- **Risorsa Collezione:** `/games`
  - Una collezione di partite.
  - Usata per i requisiti **1** (POST) e **5** (GET).
- **Risorsa Singola:** `/games/{id}`
  - Una singola partita, identificata da `{id}`.
  - Usata per i requisiti **2**, **3**, e **4**.

OPENAPI

Usiamo lo Swagger Editor per creare, validare, e testare un documento OpenAP
https://editor.swagger.io


Swagger Editor
File
Edit
Insert
Generate Server
Generate Client
About
Try our new Editor
openapi: 3.0.3
info:
title: Swagger Petstore - OpenAPI 3.0
description: |-
This is a sample Pet Store Server based on the OpenAPI 3.0 specification. You can find out more about
Swagger at https://swagger.io. In the third iteration of the pet store, we've switched to the design first approach!
You can now help us improve the API whether it's by making changes to the definition itself or to the code.
That way, with time, we can improve the API in general, and expose some of the new features in OAS3.
If you're looking for the Swagger 2.0/OAS 2.0 version of Petstore, then click [here](https://editor.swagger.io/?url=https://petstore.swagger.io/v2/swagger.yaml). Alternatively, you can load via the `Edit > Load Petstore OAS 2.0` menu option!
Some useful links:
- [The Pet Store repository](https://github.com/swagger-api/swagger-petstore)
- [The source API definition for the Pet Store](https://github.com/swagger-api/swagger-petstore/blob/master/src/main/resources/openapi.yaml)
termsOfService: http://swagger.io/terms/
contact:
email: apiteam@swagger.io
license:
name: Apache 2.0
url: http://www.apache.org/licenses/LICENSE-2.0.html
version: 1.0.11
externalDocs:
description: Find out more about Swagger
url: http://swagger.io
servers:
- url: https://petstore3.swagger.io/api/v3
tags:
- name: pet
description: Everything about your Pets
externalDocs:
description: Find out more
url: http://swagger.io
- name: store
description: Access to Petstore orders
externalDocs:
description: Find out more about our store
url: http://swagger.io
- name: user
description: Operations about user
paths:
/pet:
put:
tags:
- pet
summary: Update an existing pet
description: Update an existing pet by Id
operationId: updatePet
requestBody:
description: Update an existent pet in the store
content:
application/json:
schema:
$ref: '#/components/schemas/Pet'
Swagger Petstore - OpenAPI 3.0
1.0.11
OAS3
This is a sample Pet Store Server based on the OpenAPI 3.0 specification. You can find out more about Swagger at https://swagger.io. In the third iteration of the pet store, we've switched to the design first approach! You can now help us improve the API whether it's by making changes to the definition itself or to the code. That way, with time, we can improve the API in general, and expose some of the new features in OAS3.
If you're looking for the Swagger 2.0/OAS 2.0 version of Petstore, then click here. Alternatively, you can load via the Edit > Load Petstore OAS 2.0 menu option!
Some useful links:
The Pet Store repository
The source API definition for the Pet Store
Terms of service
Contact the developer
Apache 2.0
Find out more about Swagger
Servers
https://petstore3.swagger.io/api/v3
Authorize
pet
Everything about your Pets
Find out more
PUT /pet Update an existing pet
POST /pet Add a new pet to the store
GET /pet/findByStatus Finds Pets by status
GET /pet/findByTags Finds Pets by tags
GET /pet/{petId} Find pet by ID
POST /pet/{petId} Updates a pet in the store with form data
DELETE /pet/{petId} Deletes a pet
POST /pet/{petId}/uploadImage uploads an image

INIZIALIZZAZIONE OPENAPI

Questa è la struttura di base del documento di specifica **OpenAPI 3.0**.

```yaml
openapi: 3.0.0
info:
  title: Hi-Lo Game
  description: |
    This API allows playing hi-lo game
    and requesting the state of a game or the history
    of all the games
  version: 0.0.1
paths:
  /games:
    # Metodi GET e POST sulla collezione
    ...
  /games/{id}:
    # Metodi POST, GET, PATCH sulla singola risorsa
    ...
```

DEFINIZIONE ENDPOINT: POST /GAMES

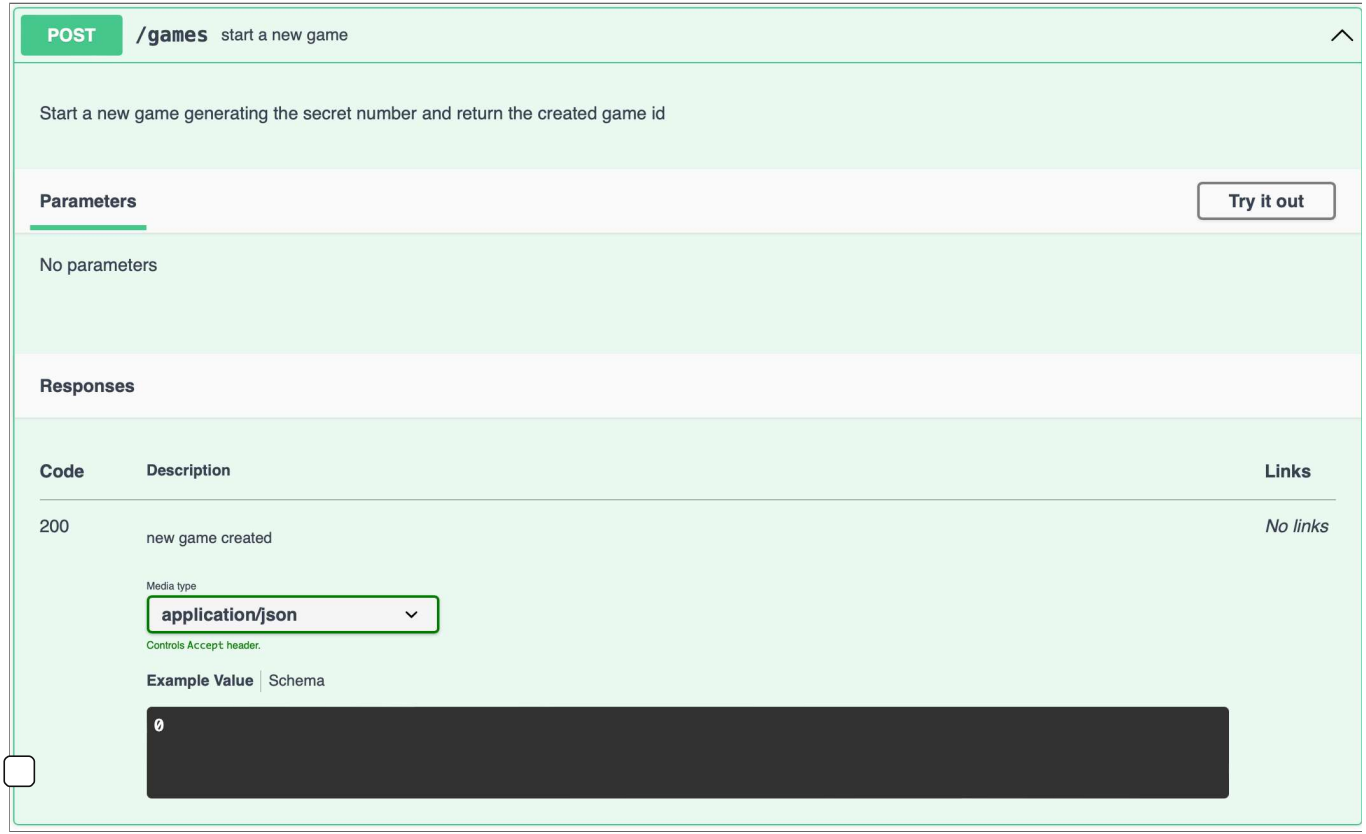
Questo metodo implementa il requisito **1: Avviare una nuova partita**.

- **Metodo:** `POST` su `/games`
- **Riassunto:** `start a new game`
- **Descrizione:** `Start a new game generating the secret number and return the created game id`
- **Risposta 200 (OK):** La risposta deve contenere l'ID della partita.
  - **Schema:** `type: integer` (l'ID della partita).

☐

```
paths:
    /games:
        post:
            summary: start a new game
            description: |
                Start a new game generating the
                 secret number and return
                 the created game id
            responses:
                "200":
                    description: new game created
                    content:
                        application/json:
                            schema:
                                type:integer
```

☐ `schema` definisce un tipo primitivo (`integer`, `string`, …), `array`, o oggetto a seconda del campo `type`


POST
/games
start a new game
Start a new game generating the secret number and return the created game id
Parameters
Try it out
No parameters
Responses
Code
Description
Links
200
new game created
No links
Media type
application/json
Controls Accept header.
Example Value
Schema
0

PARTE 2: METODI AGGIUNTIVI E RIUTILIZZO

DEFINIZIONE ENDPOINT: GET /GAMES

Questo metodo implementa il requisito **5: Ottenere la lista di tutte le partite**.

- **Metodo:** `GET` su `/games`
- **Riassunto:** `list all games`
- **Descrizione:** `Obtain the list of all games, with the final result (win/lose) and the number of guesses`
- **Risposta 200 (OK):**
  - **Schema:** `type: array` di oggetti.
  - **Proprietà Oggetto:**
    - `id`: `type: integer`
    - `outcome`: `type: string` (enum: `win`, `lose`)

```
get:
```

```
get:
    summary: list all games
    description: |
        Otain the list of all games, with
         the final result (win/lose) and
         the number of guesses
    responses:
        "200":
            description: |
                An array of objects, each containing
                the game id, the final outcome, and
                the number of guesses
            content:
                application/json:
                    schema:
                        type: array
                        items:
                            type: object
                            properties:
                                id:
                                    type: integer
                                outcome:
                                    type: string
                                    enum:
                                    - win
                                    - lose
```

```
                guesses:
                    type: integer
```

DEFINIZIONE ENDPOINT /GAMES/{ID}

```
/games/{id}:
    parameters:
    - name: id
        in: path
        required: true
        description: this is the game id
        schema:
            type: integer
            description: e.g., /games/1234
```

- [ ] un parametro nel path è sempre obbligatorio

PUT SU /GAMES/{ID}

```
put:
    summary: start or reset a game
    description: |
        Start a game with given id, generating the secret
        number; or reset an existing game, re-generating the
        secret number and zeroing the guess counter
    responses:
        "200":
            description: An existing game is reset
        "201":
            description: A new game is created
```

DEFINIZIONE ENDPOINT: POST /GAMES/{ID}

Questo metodo implementa il requisito **2: Accettare un tentativo**. È l'interazione principale del gioco.

- **Metodo:** `POST` su `/games/{id}`
- **Parametri (path):** `{id}` (`type: integer`, ID della partita).
- **Corpo Richiesta (requestBody):**
  - **Proprietà:** `guess-number` (`type: integer`).
  - **Validazione:** `minimum: 1`, `maximum: 100`.

▢

- **Risposta 200 (OK):** Dettagli del tentativo.
  - **Proprietà:**
    - `guess-count`: `type: integer` (tentativo 1 a 10).
    - `guess-outcome`: `type: string` (enum: `hi`, `lo`, `correct`).
- **Risposta 403 (Forbidden):** Gestisce il caso di *Game Over*.
  - **Proprietà:** `id`, `outcome` (`win`/`lose`), `guesses`.
- **Risposta 404 (Not Found):** Gestisce il caso di partita inesistente.

POST SU /GAMES/{ID}

```
post:
    summary: make a guess
    description: |
        Try to guess the secret number; return hi, lo,
        or correct, plus the guess count
    parameters:
        - name: guess
          in: query
          required: true
          description: the guess
          schema:
              type: integer
              minimum: 1
              maximum: 100
```

```yaml
responses:
    "200":
        description: Guess accepted, the result is in the content
        content:
            application/json:
                schema:
                    type: object
                    properties:
                        guess-count:
                            type: integer
                            minimum: 1
                            maximum: 10
                        guess-outcome:
                            type: string
                            enum:
                            - hi
                            - lo
                            - correct
```

```
"403":
    description: game over
    content:
        application/json:
            schema:
                type: object
                properties:
                    id:
                        type: integer
                    outcome:
                        type: string
                        enum:
                        - win
                        - lose
                    guesses:
                        type: integer
"404":
    description: game not found
```

DEFINIZIONE ENDPOINT: GET /GAMES/{ID}

Questo metodo implementa il requisito **4: Ottenere la lista dei tentativi**.

- **Metodo:** `GET` su `/games/{id}`
- **Riassunto:** `list guesses`
- **Descrizione:** `Return a list of all guesses in reverse chronological order, including the responses received (hi/lo/correct)`
- **Risposta 200 (OK):**
  - **Schema:** `type: array` di oggetti.
  - **Proprietà Oggetto:**
    - `guess-count`: `type: integer`.
    - `guess-outcome`: `type: string` (enum: `hi`, `lo`, `correct`).

GET SU /GAMES/{ID}

```
get:
    summary: list guesses
    description: |
        Return a list of all guesses in reverse
        chronological order, including the responses
        received (hi/lo/correct)
    responses:
        "200":
            description: Request accepted, all guesses listed in content
```

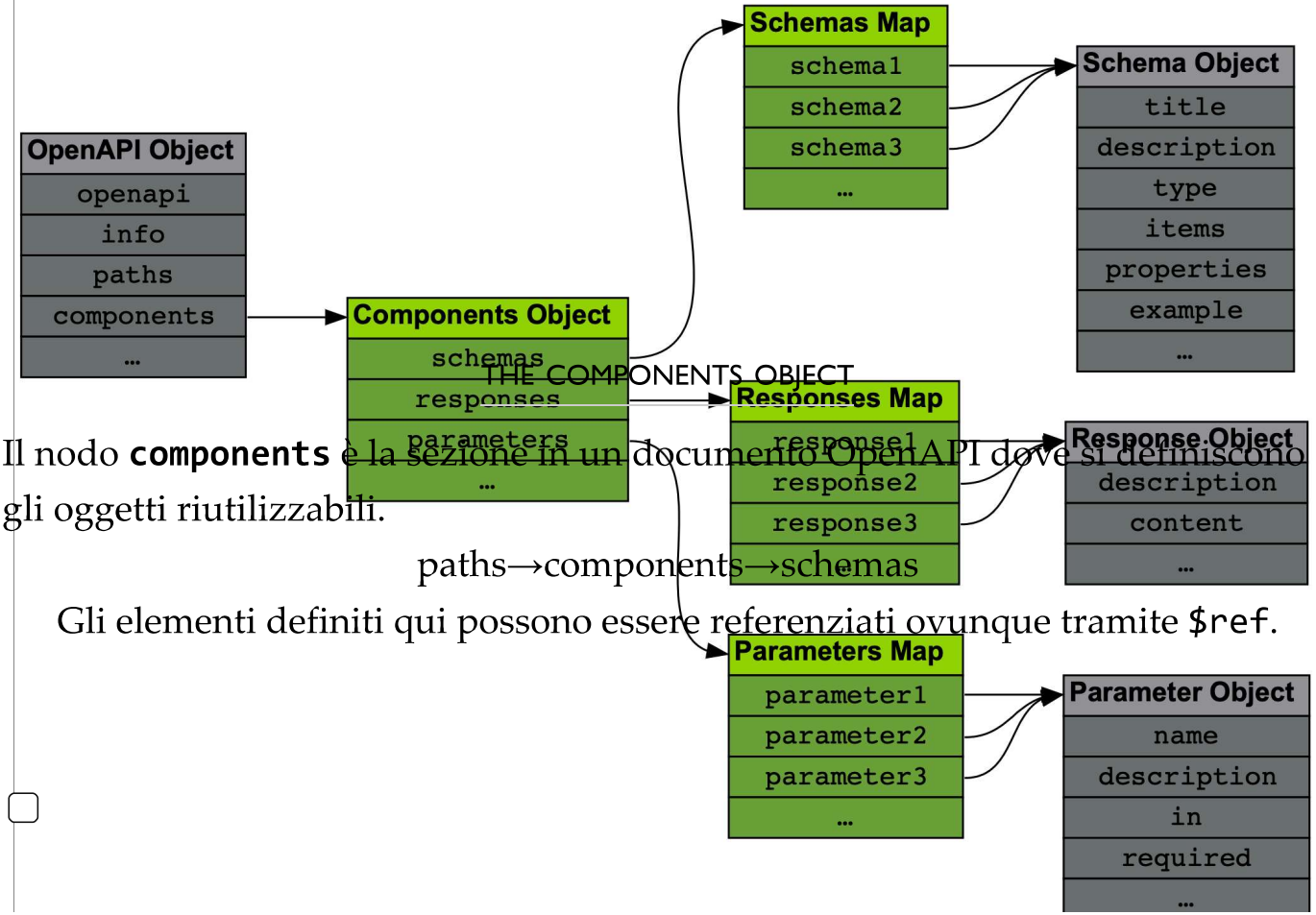
```
content:
    application/json:
        schema:
            type: array
            items:
                type: object
                properties:
                    guess-count:
                        type: integer
                        minimum: 1
                        maximum: 10
                    guess-outcome:
                        type: string
                        enum:
                        - hi
                        - lo
                        - correct
```

PARTE 3: RIUTILIZZO E SCHEMI OPENAPI

```yaml
schema:
  type: object
  properties:
    guess-count:
      type: integer
      minimum: 1
      maximum: 10
    guess-outcome:
      type: string
      enum:
      - hi
      - lo
      - correct
```

REUSING OBJECTS

- **Principio:** Possiamo sostituire gli oggetti JSON complessi all'interno delle risposte con un riferimento a un **Componente** definito centralmente.
- **Benefici:**
    - Riduce la dimensione del codice.
    - Riduce la manutenzione (modifica una sola volta).
    - Riduce gli errori (coerenza garantita).



THE COMPONENTS OBJECT

Il nodo **components** è la sezione in un documento OpenAPI dove si definiscono gli oggetti riutilizzabili.

paths→components→schemas

Gli elementi definiti qui possono essere referenziati ovunque tramite `$ref`.

GUESS-RESULT COMPONENT

Definizione dello schema riutilizzabile per il risultato di un singolo tentativo:

```yaml
components:
  schemas:
    guess-result:
      type: object
      properties:
        guess-count:
          type: integer
          minimum: 1
          maximum: 10
        guess-outcome:
          type: string
          enum:
            - hi
            - lo
            - correct
```

ESEMPIO DI UTILIZZO ($REF)

Questo mostra come l'endpoint `GET /games/{id}/guesses` ora utilizza il componente `guess-result`:

```
responses:
  "200":
    description: Request accepted, all guesses listed in content
    content:
      application/json:
        schema:
          type: array
          items:
            $ref: "#/components/schemas/guess-result" # Riferimento al
componente
```

```yaml
components:
  schemas:
    guess-result:
      type: object
      properties:
        guess-count:
          type: integer
          minimum: 1
          maximum: 10
        guess-outcome:
          type: string
          enum:
          - hi
          - lo
          - correct
```

FINAL-RESULT COMPONENT

Definizione dello schema riutilizzabile per il risultato finale di una partita (usato in `GET /games` e `POST /games/{id}/guesses` - errore 403):

```yaml
final-result:
  type: object
  properties:
    id:
      type: integer
    outcome:
      type: string
      enum:
        - win
        - lose
    guesses:
      type: integer
```

- [ ]

LINK UTILI

- https://www.openapis.org
- https://oai.github.io/Documentation/
- http://gamificationlab.uniroma1.it/notes/hilo-game-final.yaml